

CHAPTER 6

INHERITANCE

1. What is Inheritance?

Inheritance is an OOP concept in which one class acquires the properties and methods of another class.

In simple words:

Inheritance allows one class to reuse the code of another class.

The existing class is called the Parent/Superclass, and the new class is called the Child/Subclass.

Example

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

Here:

- Animal → Parent class
- Dog → Child class
- extends → Keyword used for inheritance

- eat() → Inherited method
- bark() → Dog's own method

Using the classes

```
class Main {  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.eat();  
        d.bark();  
    }  
}
```

Output

Animal eats

Dog barks

The Dog class can use eat() even though eat() is written inside Animal.

2. Advantages of Inheritance

Inheritance provides several advantages.

1. Code Reusability

We can reuse the code of the parent class instead of writing the same code again.

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

```
}  
}
```

```
class Dog extends Animal {  
}
```

Dog automatically gets the eat() method.

2. Reduces Code Duplication

Common methods can be written once in the parent class and reused by multiple child classes.

3. Easy Maintenance

If common functionality needs to be changed, we can modify the parent class instead of modifying every child class.

4. Supports Method Overriding

Inheritance allows a child class to provide its own version of a parent method.

5. Supports Polymorphism

Inheritance is one of the important foundations for **runtime polymorphism** in Java.

3. The extends Keyword

The extends keyword is used to create inheritance between classes.

Syntax

```
class Child extends Parent {  
    // body  
}
```

Example

```
class Employee {  
    void work() {  
        System.out.println("Employee is working");  
    }  
}
```

```
class Developer extends Employee {  
    void code() {  
        System.out.println("Developer is coding");  
    }  
}
```

Here:

class Developer extends Employee
means Developer inherits from Employee.

Example usage

```
Developer d = new Developer();
```

```
d.work();
```

```
d.code();
```

Output:

Employee is working

Developer is coding

Important point

A Java class can extend **only one class**.

```
class Dog extends Animal {  
}
```

This is valid.

But:

```
class Dog extends Animal, Mammal {  
}
```

is **not valid** in Java.

4. Types of Inheritance

The major types of inheritance are:

1. **Single Inheritance**
2. **Multilevel Inheritance**
3. **Hierarchical Inheritance**
4. **Multiple Inheritance** – not supported through classes in Java
5. **Hybrid Inheritance** – not directly supported through classes

4.1 Single Inheritance

One child class inherits from one parent class.

Parent

↓

Child

Example:

```
class Animal {
```

```
void eat() {  
    System.out.println("Eating");  
}  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

Here:

Animal

↓

Dog

Dog inherits from Animal.

4.2 Multilevel Inheritance

Inheritance occurs in multiple levels.

Grandparent

↓

Parent

↓

Child

Example:

```
class Animal {
```

```
void eat() {  
    System.out.println("Eating");  
}  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

```
class Puppy extends Dog {  
    void play() {  
        System.out.println("Playing");  
    }  
}
```

Now Puppy can access methods from both Dog and Animal.

```
Puppy p = new Puppy();
```

```
p.eat();
```

```
p.bark();
```

```
p.play();
```

4.3 Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

Animal

/ \

/ \

Dog Cat

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

```
class Cat extends Animal {  
    void meow() {  
        System.out.println("Meowing");  
    }  
}
```

Both Dog and Cat can use eat().

4.4 Multiple Inheritance

Multiple inheritance means one child has **more than one parent**.

Parent 1 Parent 2

\ /

\ /

Child

Java **does not support multiple inheritance using classes**.

For example, this is not allowed:

```
class C extends A, B {  
  
}
```

Why?

It can create ambiguity.

Suppose both A and B have a method called show().

If C inherits both, Java would have difficulty deciding which show() should be used.

This is sometimes called the **Diamond Problem**.

However, Java can achieve multiple inheritance of type through **interfaces**.

5. super Keyword

The super keyword refers to the **immediate parent class object**.

It is mainly used for three purposes:

1. Access parent class variable
2. Call parent class method
3. Call parent class constructor

5.1 super to Access Parent Variable

```
class Animal {  
    String name = "Animal";  
}  
  
class Dog extends Animal {  
    String name = "Dog";  
  
    void display() {  
        System.out.println(name);  
        System.out.println(super.name);  
    }  
}
```

Output:

Dog

Animal

Here:

name

refers to the child variable.

super.name

refers to the parent variable.

6. super to Call Parent Method

Suppose both parent and child have a method with the same name.

```
class Animal {
```

```
void sound() {  
    System.out.println("Animal makes sound");  
}  
}
```

```
class Dog extends Animal {
```

```
    void sound() {  
        System.out.println("Dog barks");  
    }
```

```
    void display() {  
        sound();  
        super.sound();  
    }  
}
```

Output:

Dog barks

Animal makes sound

Here:

sound();

calls the child method.

super.sound();

calls the parent method.

7. super() – Calling Parent Constructor

super() is used to call the constructor of the parent class.

```
class Animal {  
  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
}
```

```
class Dog extends Animal {  
  
    Dog() {  
        super();  
        System.out.println("Dog constructor");  
    }  
}
```

Output:

Animal constructor

Dog constructor

Important

If we don't write super() explicitly, Java automatically inserts a call to the parent's no-argument constructor, provided such a constructor is available.

8. Forms of super

There are mainly **three forms**:

1. Parent variable

```
super.variable;
```

Example:

```
super.name;
```

2. Parent method

```
super.method();
```

Example:

```
super.display();
```

3. Parent constructor

```
super();
```

or

```
super(value);
```

9. Interface Keyword

An **interface** is a blueprint that contains method declarations and constants, and it is used to define a contract that implementing classes must follow.

The keyword used to create an interface is:

```
interface
```

A class uses the keyword:

```
implements
```

to implement an interface.

Example

```
interface Animal {
```

```
void sound();  
  
}  
  
class Dog implements Animal {  
  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Here:

- Animal → Interface
- Dog → Class
- implements → Keyword
- sound() → Implemented method

Main method

```
class Main {  
  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output:

Dog barks

Why interfaces are useful?

Interfaces are useful for:

- Abstraction
- Achieving multiple inheritance of type
- Providing a common contract
- Loose coupling

Multiple interfaces

A class can implement multiple interfaces.

```
interface A {  
    void showA();  
}
```

```
interface B {  
    void showB();  
}
```

```
class C implements A, B {
```

```
    public void showA() {  
        System.out.println("A");  
    }
```

```
    public void showB() {  
        System.out.println("B");  
    }  
}
```

This is allowed in Java.

10. final Keyword

The final keyword is used to restrict changes.

It can be used with:

1. Variables
 2. Methods
 3. Classes
-

10.1 Final Variable

A final variable cannot be reassigned after initialization.

```
final int age = 20;
```

This is not allowed:

```
age = 25;
```

It gives a compilation error.

Example

```
class Student {  
    final int rollNo = 101;  
}
```

10.2 Final Method

A final method cannot be overridden by a child class.

```
class Animal {  
  
    final void eat() {
```



```
        System.out.println("Eating");
    }
}
```

A child class cannot override eat().

10.3 Final Class

A final class cannot be inherited.

```
final class Animal {
}
```

This is not allowed:

```
class Dog extends Animal {
}
```

because Animal is final.

Simple memory trick

final variable → cannot change

final method → cannot override

final class → cannot inherit

11. instanceof Keyword

Your image says "**Instant Keyword**", but the Java keyword is actually **instanceof**.

instanceof is used to check whether an object belongs to a particular class or interface type.

Syntax

object instanceof ClassName

It returns either:

true

or

false

Example

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```

```
class Main {  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        System.out.println(d instanceof Dog);  
        System.out.println(d instanceof Animal);  
    }  
}
```

Output:

true

true

Why is the second one also true?

Because Dog inherits from Animal.

So a Dog object is also considered an Animal.

12. instanceof with if

It is commonly used with an if statement.

```
class Animal {  
  
}  
  
class Dog extends Animal {  
  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        Animal a = new Dog();  
  
        if (a instanceof Dog) {  
            System.out.println("The object is a Dog");  
        }  
    }  
}
```

Output:

The object is a Dog

This is useful when working with **inheritance and polymorphism**.

13. Complete Example Using Inheritance

Here is a beginner-level project-oriented example.

Employee Management

```
class Employee {
```

```
    String name;
```

```
    double salary;
```

```
    Employee(String name, double salary) {
```

```
        this.name = name;
```

```
        this.salary = salary;
```

```
    }
```

```
    void displayEmployee() {
```

```
        System.out.println("Name: " + name);
```

```
        System.out.println("Salary: " + salary);
```

```
    }
```

```
}
```

```
class Developer extends Employee {
```

```
    String language;
```

```
    Developer(String name, double salary, String language) {
```

```
        super(name, salary);
```

```
        this.language = language;
```

```
    }
```

```
void displayDeveloper() {  
    displayEmployee();  
    System.out.println("Programming Language: " + language);  
}  
}
```

```
class Main {  
  
    public static void main(String[] args) {  
  
        Developer d = new Developer(  
            "Arun",  
            30000,  
            "Java"  
        );  
  
        d.displayDeveloper();  
    }  
}
```

Output

Name: Arun

Salary: 30000.0

Programming Language: Java

Inheritance



extends → Class inheritance



super → Parent class access



interface → Contract / abstraction



implements → Implement interface



final → Restriction



instanceof → Check object type.